

Departmental Requisition Form System

Full-Stack Web Application — Technical Documentation

Platform: Django (Python) · SQLite · Tailwind CSS

Version 1.0

Date: May 2026

1. Project Overview

This document describes a full-stack web application that implements a departmental requisition form system. Each department logs in with its own credentials, fills out a daily requisition form listing fixed items along with the desired unit and quantity, and can later edit its submission.

The original specification requested Node.js + Express.js + EJS. This document re-implements the same functional requirements on the Django (Python) platform, while preserving the user-facing behavior, database design, workflow, and UI structure. Several enhancements are added to match production-grade Django conventions (CSRF protection, Django's built-in auth, class-based views, migrations, environment configuration, etc.).

1.1 Technology Stack

Layer	Technology	Purpose
Backend Framework	Django 5.x (Python 3.11+)	MVT architecture, ORM, routing
Database	SQLite 3	Lightweight relational DB (default Django DB)
Frontend Styling	Tailwind CSS 3.x	Utility-first responsive UI
Templating	Django Templates (DTL)	Server-side rendered pages
Authentication	Django auth + sessions	Session-based login with hashed passwords
Forms / Validation	Django Forms + ModelForms	Server-side validation
Static / Build	django-tailwind + Node.js	Tailwind compilation pipeline
Password Hashing	PBKDF2 (Django default)	Equivalent to bcrypt; configurable
Messaging	django.contrib.messages	Flash messages for success/errors

1.2 Key Features

- Department-based authentication using Django's session framework.
- Fixed master list of requisition items shared across all departments.
- Form layout: item name on the left, unit dropdown in the middle, quantity on the right.
- Units supported: pcs, rolls, kg, litres, boxes, packets, dozen, metres, grams, millilitres.
- One submission per department per day enforced at the database level (UniqueConstraint).
- Edit existing submissions; updated_at timestamp tracks the latest change.
- Responsive UI built with Tailwind utility classes.
- CSRF protection on every POST request (Django default).
- Flash messages for success and error states.
- Admin panel via django.contrib.admin to manage items, users, and submissions.
- Seed management command for sample items and a test department.

1.3 Additional Enhancements

Beyond the original specification, this Django version adds the following improvements:

- Django admin interface for non-technical management of items and users.
- Custom management commands (`seed_items`, `create_test_user`) for repeatable setup.
- Soft-delete (`is_active`) flag on the Item model so retired items disappear from the form without breaking historical submissions.
- Submission history page listing every previous requisition with status and timestamps.
- Pagination on submission history (Django Paginator).
- Server-side validation: quantity must be a positive number, unit must be one of the allowed choices.
- Environment-based settings via `python-dotenv` for `SECRET_KEY` and `DEBUG`.
- Print-friendly view of a submitted form for departmental records.
- Class-based views (`LoginRequiredMixin`) for clean controller separation.

2. Architecture (MVT in Django)

Django follows the Model-View-Template (MVT) pattern, which is conceptually equivalent to MVC:

MVC (typical)	Django MVT equivalent	Responsibility
Model	Model (models.py)	Database schema and business logic
View / Controller	View (views.py) + URLs	Request handling, orchestration
View / Template	Template (.html)	Presentation layer

2.1 Request Lifecycle

1. Browser sends HTTP request to Django.
2. URL dispatcher (urls.py) matches the path to a view.
3. Middleware (SessionMiddleware, AuthenticationMiddleware, CSRF) runs.
4. View executes — queries models, validates forms, builds context.
5. Template renders HTML with the context.
6. Response returned to the browser.

3. Folder Structure

Recommended project layout. The Django project is named `requisition_project` and the main app is named `requisitions`.

```
requisition_project/
├── manage.py
├── requirements.txt
├── .env                                # SECRET_KEY, DEBUG, etc.
├── package.json                       # Tailwind build scripts
├── tailwind.config.js
├── postcss.config.js
├── requisition_project/               # Project settings package
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── wsgi.py
│   └── asgi.py
├── requisitions/                     # Main app
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── forms.py
│   ├── models.py
│   ├── urls.py
│   ├── views.py
│   ├── decorators.py
│   ├── migrations/
│   │   └── __init__.py
│   ├── management/
│   │   └── commands/
│   │       ├── seed_items.py
│   │       └── create_test_user.py
│   └── templates/
│       └── requisitions/
│           ├── base.html
│           ├── login.html
│           ├── dashboard.html
│           ├── form_create.html
│           ├── form_edit.html
│           ├── success.html
│           └── history.html
├── static/                           # Compiled CSS (output)
│   └── css/
│       └── tailwind.css
├── theme/                             # Tailwind source (django-tailwind)
│   ├── static_src/
│   │   └── src/
│   │       └── styles.css
```

```
├── package.json
├── tailwind.config.js
├── postcss.config.js
├── templates/
└── db.sqlite3 # SQLite database (auto-created)
```

4. Installation & Package Setup

4.1 Prerequisites

- Python 3.11 or higher
- pip and venv
- Node.js 18+ and npm (for Tailwind compilation)
- Git (optional)

4.2 Create Project Directory & Virtual Environment

```
# Linux / macOS
mkdir requisition_project && cd requisition_project
python3 -m venv venv
source venv/bin/activate

# Windows (PowerShell)
mkdir requisition_project; cd requisition_project
python -m venv venv
venv\Scripts\Activate.ps1
```

4.3 Install Python Dependencies

```
pip install --upgrade pip
pip install django==5.0.*
pip install django-tailwind
pip install django-browser-reload      # auto-refresh during dev
pip install python-dotenv              # env variable management
pip install Pillow                    # optional, for image fields
pip freeze > requirements.txt
```

4.4 Create Django Project & App

```
django-admin startproject requisition_project .
python manage.py startapp requisitions
python manage.py tailwind init theme    # creates the 'theme' app for Tailwind
```

4.5 requirements.txt (final)

```
Django>=5.0,<5.1
django-tailwind>=3.8
django-browser-reload>=1.12
python-dotenv>=1.0
Pillow>=10.0
```

5. Settings Configuration

5.1 settings.py (excerpt)

```
from pathlib import Path
from dotenv import load_dotenv
import os

BASE_DIR = Path(__file__).resolve().parent.parent
load_dotenv(BASE_DIR / ".env")

SECRET_KEY = os.getenv("SECRET_KEY", "dev-insecure-key-change-me")
DEBUG = os.getenv("DEBUG", "True") == "True"
ALLOWED_HOSTS = ["*"]

INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",

    # Third-party
    "tailwind",
    "theme",
    "django_browser_reload",

    # Local
    "requisitions",
]

MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
    "django_browser_reload.middleware.BrowserReloadMiddleware",
]

ROOT_URLCONF = "requisition_project.urls"
WSGI_APPLICATION = "requisition_project.wsgi.application"

TEMPLATES = [{
    "BACKEND": "django.template.backends.django.DjangoTemplates",
    "DIRS": [BASE_DIR / "templates"],
    "APP_DIRS": True,
    "OPTIONS": {
        "context_processors": [
            "django.template.context_processors.debug",
            "django.template.context_processors.request",
```



```

        "django.contrib.auth.context_processors.auth",
        "django.contrib.messages.context_processors.messages",
    ],
},
}]

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}

# Tailwind
TAILWIND_APP_NAME = "theme"
INTERNAL_IPS = ["127.0.0.1"]
NPM_BIN_PATH = "npm" # or absolute path on Windows

# Auth
LOGIN_URL = "/login/"
LOGIN_REDIRECT_URL = "/dashboard/"
LOGOUT_REDIRECT_URL = "/login/"

# Sessions
SESSION_COOKIE_AGE = 60 * 60 * 8 # 8 hours
SESSION_EXPIRE_AT_BROWSER_CLOSE = False

# Password hashing – PBKDF2 is the Django default (equivalent strength to bcrypt)
PASSWORD_HASHERS = [
    "django.contrib.auth.hashers.PBKDF2PasswordHasher",
    "django.contrib.auth.hashers.BCryptSHA256PasswordHasher",
]

STATIC_URL = "static/"
STATICFILES_DIRS = [BASE_DIR / "static"]
DEFAULT_AUTO_FIELD = "django.db.models.BigAutoField"

```

5.2 .env File

```

SECRET_KEY=replace-this-with-a-long-random-string
DEBUG=True

```

6. Database Schema

The schema mirrors the original specification, with Django conventions applied (auto-managed primary keys, timestamps, and a UniqueConstraint for one-submission-per-day).

6.1 Tables Overview

Table	Purpose
auth_user (Django built-in)	Stores username, hashed password, email, etc.
requisitions_department	Maps each Django user to a department name.
requisitions_item	Master list of fixed items available on the form.
requisitions_formsubmission	One row per department per day.
requisitions_formsubmissionitem	Per-item line: unit + quantity per submission.

6.2 models.py

```
from django.db import models
from django.contrib.auth.models import User
from django.core.validators import MinValueValidator
from django.utils import timezone

class Department(models.Model):
    """One-to-one link between Django's auth User and a department name."""
    user = models.OneToOneField(User, on_delete=models.CASCADE, related_name="department")
    department_name = models.CharField(max_length=120, unique=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.department_name

class Item(models.Model):
    """Fixed item master list, shared across all departments."""
    item_name = models.CharField(max_length=150, unique=True)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["item_name"]

    def __str__(self):
        return self.item_name

class FormSubmission(models.Model):
    """One submission per user per day."""
```

```

user = models.ForeignKey(User, on_delete=models.CASCADE, related_name="submissions")
submitted_at = models.DateTimeField(auto_now_add=True)
updated_at = models.DateTimeField(auto_now=True)
submission_date = models.DateField(default=timezone.localdate)

class Meta:
    ordering = ["-submitted_at"]
    constraints = [
        models.UniqueConstraint(
            fields=["user", "submission_date"],
            name="one_submission_per_user_per_day",
        )
    ]

def __str__(self):
    return f"{self.user.username} - {self.submission_date}"

class FormSubmissionItem(models.Model):
    UNIT_CHOICES = [
        ("pcs", "pcs"),
        ("rolls", "rolls"),
        ("kg", "kg"),
        ("g", "g"),
        ("litres", "litres"),
        ("ml", "ml"),
        ("boxes", "boxes"),
        ("packets", "packets"),
        ("dozen", "dozen"),
        ("metres", "metres"),
    ]

    submission = models.ForeignKey(
        FormSubmission, on_delete=models.CASCADE, related_name="line_items"
    )
    item = models.ForeignKey(Item, on_delete=models.PROTECT)
    unit = models.CharField(max_length=20, choices=UNIT_CHOICES, default="pcs")
    quantity = models.DecimalField(
        max_digits=10, decimal_places=2,
        validators=[MinValueValidator(0)],
    )

    class Meta:
        unique_together = ("submission", "item")
        ordering = ["item__item_name"]

    def __str__(self):
        return f"{self.item.item_name}: {self.quantity} {self.unit}"

```

6.3 Create & Apply Migrations

```

python manage.py makemigrations requisitions
python manage.py migrate

```


7. Seed Data (Management Commands)

Two custom management commands seed the items table and create a test department user.

7.1 requisitions/management/commands/seed_items.py

```
from django.core.management.base import BaseCommand
from requisitions.models import Item

DEFAULT_ITEMS = [
    "A4 Paper", "Pen (Blue)", "Pen (Black)", "Pencil",
    "Notebook", "Stapler Pins", "Cello Tape", "Glue Stick",
    "Marker (Permanent)", "Whiteboard Marker", "Sticky Notes",
    "File Folder", "Printer Ink (Black)", "Printer Ink (Color)",
    "Hand Sanitizer", "Tissue Box", "Disinfectant", "Rice",
    "Sugar", "Tea Bags", "Coffee Powder", "Milk", "Cooking Oil",
]

class Command(BaseCommand):
    help = "Seeds the Item table with a default master list."

    def handle(self, *args, **options):
        created = 0
        for name in DEFAULT_ITEMS:
            obj, was_created = Item.objects.get_or_create(item_name=name)
            if was_created:
                created += 1
        self.stdout.write(self.style.SUCCESS(
            f"Seed complete. {created} new items inserted, "
            f"{len(DEFAULT_ITEMS) - created} already existed."
        ))
```

7.2 requisitions/management/commands/create_test_user.py

```
from django.core.management.base import BaseCommand
from django.contrib.auth.models import User
from requisitions.models import Department

class Command(BaseCommand):
    help = "Creates a test department user."

    def handle(self, *args, **options):
        username = "admin_dept"
        password = "admin123"
        dept_name = "Administration"

        if User.objects.filter(username=username).exists():
            self.stdout.write(self.style.WARNING(f"User '{username}' already exists."))
            return
```

```
user = User.objects.create_user(username=username, password=password)
Department.objects.create(user=user, department_name=dept_name)
self.stdout.write(self.style.SUCCESS(
    f"Test user created → username: {username} password: {password} "
    f"department: {dept_name}"
))
```

7.3 Run Seeds

```
python manage.py seed_items
python manage.py create_test_user
```

8. Forms (forms.py)

```
from django import forms
from django.contrib.auth.forms import AuthenticationForm
from .models import FormSubmissionItem, Item

class StyledAuthForm(AuthenticationForm):
    """Adds Tailwind classes to Django's default login form."""
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        css = "w-full border border-gray-300 rounded-lg px-4 py-2 focus:ring-2
focus:ring-blue-500"
        self.fields["username"].widget.attrs.update({"class": css, "placeholder":
"Username"})
        self.fields["password"].widget.attrs.update({"class": css, "placeholder":
"Password"})

class LineItemForm(forms.Form):
    """One row of the requisition table."""
    item_id = forms.IntegerField(widget=forms.HiddenInput())
    unit = forms.ChoiceField(
        choices=FormSubmissionItem.UNIT_CHOICES,
        widget=forms.Select(attrs={"class": "border rounded-md px-2 py-1 w-full"}),
        required=False,
    )
    quantity = forms.DecimalField(
        max_digits=10, decimal_places=2, min_value=0, required=False,
        widget=forms.NumberInput(attrs={
            "class": "border rounded-md px-2 py-1 w-full",
            "step": "0.01", "min": "0", "placeholder": "0",
        })),
    )
```

9. Views (Controllers)

9.1 requisitions/views.py

```
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.decorators import login_required
from django.contrib import messages
from django.shortcuts import render, redirect, get_object_or_404
from django.utils import timezone
from django.db import transaction
from django.core.paginator import Paginator

from .models import Item, FormSubmission, FormSubmissionItem
from .forms import StyledAuthForm

# ----- Auth -----
def login_view(request):
    if request.user.is_authenticated:
        return redirect("dashboard")
    if request.method == "POST":
        form = StyledAuthForm(request, data=request.POST)
        if form.is_valid():
            user = form.get_user()
            login(request, user)
            messages.success(request, f"Welcome, {user.username}!")
            return redirect("dashboard")
        messages.error(request, "Invalid credentials.")
    else:
        form = StyledAuthForm(request)
    return render(request, "requisitions/login.html", {"form": form})

def logout_view(request):
    logout(request)
    messages.info(request, "You have been logged out.")
    return redirect("login")

# ----- Dashboard -----
@login_required
def dashboard(request):
    today = timezone.localdate()
    today_submission = FormSubmission.objects.filter(
        user=request.user, submission_date=today
    ).first()

    qs = FormSubmission.objects.filter(user=request.user).order_by("-submission_date")
    paginator = Paginator(qs, 10)
    page = paginator.get_page(request.GET.get("page"))

    return render(request, "requisitions/dashboard.html", {
        "today_submission": today_submission,
        "page_obj": page,
```



```

        "today": today,
    })

# ----- Create -----
@login_required
def create_form(request):
    today = timezone.localdate()

    # Prevent duplicate submission for the same day.
    existing = FormSubmission.objects.filter(user=request.user,
submission_date=today).first()
    if existing:
        messages.info(request, "You already submitted today. Redirecting to edit.")
        return redirect("edit_form", submission_id=existing.id)

    items = Item.objects.filter(is_active=True)

    if request.method == "POST":
        try:
            with transaction.atomic():
                submission = FormSubmission.objects.create(
                    user=request.user, submission_date=today
                )
                any_line = False
                for item in items:
                    qty_raw = request.POST.get(f"quantity_{item.id}", "").strip()
                    unit = request.POST.get(f"unit_{item.id}", "pcs")
                    if qty_raw == "":
                        continue
                    try:
                        qty = float(qty_raw)
                    except ValueError:
                        raise ValueError(f"Invalid quantity for {item.item_name}")
                    if qty < 0:
                        raise ValueError(f"Quantity must be ≥ 0 for {item.item_name}")
                    if qty == 0:
                        continue
                    FormSubmissionItem.objects.create(
                        submission=submission, item=item, unit=unit, quantity=qty
                    )
                    any_line = True
                if not any_line:
                    raise ValueError("At least one item must have a quantity.")
            messages.success(request, "Requisition submitted successfully.")
            return redirect("success", submission_id=submission.id)
        except ValueError as e:
            messages.error(request, str(e))

    return render(request, "requisitions/form_create.html", {
        "items": items,
        "units": FormSubmissionItem.UNIT_CHOICES,
    })

```

```

# ----- Edit -----
@login_required
def edit_form(request, submission_id):
    submission = get_object_or_404(FormSubmission, id=submission_id, user=request.user)
    items = Item.objects.filter(is_active=True)
    existing = {li.item_id: li for li in submission.line_items.all()}

    if request.method == "POST":
        try:
            with transaction.atomic():
                submission.line_items.all().delete()
                for item in items:
                    qty_raw = request.POST.get(f"quantity_{item.id}", "").strip()
                    unit = request.POST.get(f"unit_{item.id}", "pcs")
                    if qty_raw == "":
                        continue
                    qty = float(qty_raw)
                    if qty < 0:
                        raise ValueError(f"Negative quantity for {item.item_name}")
                    if qty == 0:
                        continue
                    FormSubmissionItem.objects.create(
                        submission=submission, item=item, unit=unit, quantity=qty
                    )
                submission.save() # bump updated_at
            messages.success(request, "Submission updated.")
            return redirect("success", submission_id=submission.id)
        except ValueError as e:
            messages.error(request, str(e))

    return render(request, "requisitions/form_edit.html", {
        "submission": submission,
        "items": items,
        "existing": existing,
        "units": FormSubmissionItem.UNIT_CHOICES,
    })

# ----- Success -----
@login_required
def success_view(request, submission_id):
    submission = get_object_or_404(FormSubmission, id=submission_id, user=request.user)
    return render(request, "requisitions/success.html", {"submission": submission})

```

10. URL Routing

10.1 Project urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("requisitions.urls")),
    path("__reload__/", include("django_browser_reload.urls")),
]
```

10.2 App urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.login_view, name="login"),
    path("login/", views.login_view, name="login"),
    path("logout/", views.logout_view, name="logout"),
    path("dashboard/", views.dashboard, name="dashboard"),
    path("form/new/", views.create_form, name="create_form"),
    path("form/<int:submission_id>/edit/",
          views.edit_form, name="edit_form"),
    path("form/<int:submission_id>/success/",
          views.success_view, name="success"),
]
```

11. Tailwind CSS Setup

11.1 Initialize the Tailwind app

```
# Already done during installation; re-run if needed:
python manage.py tailwind init theme
python manage.py tailwind install
python manage.py tailwind start    # watch mode (during development)
# Production build:
python manage.py tailwind build
```

11.2 theme/static_src/tailwind.config.js

```
module.exports = {
  content: [
    "../templates/**/*.html",
    "../../templates/**/*.html",
    "../../**/templates/**/*.html",
  ],
  theme: { extend: {} },
  plugins: [require("@tailwindcss/forms")],
}
```

11.3 Loading Tailwind in templates

```
{% load static tailwind_tags %}
<head>
  {% tailwind_css %}
</head>
```

12. Templates (Views)

12.1 base.html

```
{% load static tailwind_tags %}
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>{% block title %}Requisition System{% endblock %}</title>
  {% tailwind_css %}
</head>
<body class="bg-gray-50 min-h-screen flex flex-col">

<nav class="bg-blue-700 text-white shadow">
  <div class="max-w-6xl mx-auto px-4 py-3 flex justify-between items-center">
    <a href="{% url 'dashboard' %}" class="font-bold text-lg">Requisition System</a>
    <div class="text-sm">
      {% if user.is_authenticated %}
        <span class="mr-4">{{ user.username }} ({{ user.department.department_name
      }})</span>
        <a href="{% url 'logout' %}" class="bg-red-500 hover:bg-red-600 px-3 py-1
rounded">Logout</a>
      {% endif %}
    </div>
  </div>
</nav>

{% if messages %}
  <div class="max-w-6xl mx-auto w-full px-4 mt-4 space-y-2">
    {% for msg in messages %}
      <div class="px-4 py-2 rounded
        {% if msg.tags == 'success' %}bg-green-100 text-green-800 border border-green-300
        {% elif msg.tags == 'error' %}bg-red-100 text-red-800 border border-red-300
        {% else %}bg-blue-100 text-blue-800 border border-blue-300{% endif %}">
        {{ msg }}
      </div>
    {% endfor %}
  </div>
{% endif %}

<main class="max-w-6xl mx-auto w-full px-4 py-6 flex-grow">
  {% block content %}{% endblock %}
</main>

<footer class="bg-gray-100 text-center text-xs text-gray-500 py-3">
  &copy; 2026 Requisition System
</footer>
</body>
</html>
```

12.2 login.html

```

{% extends "requisitions/base.html" %}
{% block title %}Login{% endblock %}
{% block content %}
<div class="max-w-md mx-auto mt-12 bg-white p-8 shadow-md rounded-lg">
  <h1 class="text-2xl font-bold mb-6 text-center text-gray-800">Department Login</h1>
  <form method="post" class="space-y-4">
    {% csrf_token %}
    <div>
      <label class="block text-sm font-medium mb-1">Username</label>
      {{ form.username }}
    </div>
    <div>
      <label class="block text-sm font-medium mb-1">Password</label>
      {{ form.password }}
    </div>
    <button type="submit"
      class="w-full bg-blue-600 hover:bg-blue-700 text-white font-semibold py-2
rounded-lg">
      Sign In
    </button>
  </form>
</div>
{% endblock %}

```

12.3 dashboard.html

```

{% extends "requisitions/base.html" %}
{% block title %}Dashboard{% endblock %}
{% block content %}
<div class="flex justify-between items-center mb-6">
  <h1 class="text-2xl font-bold">Dashboard</h1>
  {% if today_submission %}
    <a href="{% url 'edit_form' today_submission.id %}"
      class="bg-yellow-500 hover:bg-yellow-600 text-white px-4 py-2 rounded">
      Edit Today's Requisition
    </a>
  {% else %}
    <a href="{% url 'create_form' %}"
      class="bg-green-600 hover:bg-green-700 text-white px-4 py-2 rounded">
      + New Requisition
    </a>
  {% endif %}
</div>

<h2 class="text-lg font-semibold mb-2">Submission History</h2>
<div class="bg-white shadow rounded overflow-hidden">
  <table class="min-w-full text-sm">
    <thead class="bg-gray-100">
      <tr>
        <th class="text-left px-4 py-2">Date</th>
        <th class="text-left px-4 py-2">Submitted At</th>
        <th class="text-left px-4 py-2">Last Updated</th>
        <th class="text-left px-4 py-2">Action</th>
      </tr>
    </thead>
  </table>
</div>

```

```

        </tr>
    </thead>
    <tbody>
        {% for s in page_obj %}
            <tr class="border-t">
                <td class="px-4 py-2">{{ s.submission_date }}</td>
                <td class="px-4 py-2">{{ s.submitted_at|date:"d M Y H:i" }}</td>
                <td class="px-4 py-2">{{ s.updated_at|date:"d M Y H:i" }}</td>
                <td class="px-4 py-2">
                    <a href="{% url 'edit_form' s.id %}" class="text-blue-600
hover:underline">Edit</a>
                </td>
            </tr>
        {% empty %}
            <tr><td colspan="4" class="px-4 py-4 text-center text-gray-500">No submissions
yet.</td></tr>
        {% endfor %}
    </tbody>
</table>
</div>
{% endblock %}

```

12.4 form_create.html

```

{% extends "requisitions/base.html" %}
{% block title %}New Requisition{% endblock %}
{% block content %}
<h1 class="text-2xl font-bold mb-4">New Requisition – {% now "d M Y" %}</h1>

<form method="post" class="bg-white shadow rounded-lg overflow-hidden">
    {% csrf_token %}
    <table class="min-w-full text-sm">
        <thead class="bg-blue-700 text-white">
            <tr>
                <th class="text-left px-4 py-2 w-1/2">Item</th>
                <th class="text-left px-4 py-2">Unit</th>
                <th class="text-left px-4 py-2">Quantity</th>
            </tr>
        </thead>
        <tbody>
            {% for item in items %}
                <tr class="border-t">
                    <td class="px-4 py-2 font-medium">{{ item.item_name }}</td>
                    <td class="px-4 py-2">
                        <select name="unit_{{ item.id }}"
                            class="border rounded px-2 py-1 w-full">
                            {% for value,label in units %}
                                <option value="{{ value }}">{{ label }}</option>
                            {% endfor %}
                        </select>
                    </td>
                    <td class="px-4 py-2">
                        <input type="number" name="quantity_{{ item.id }}" min="0" step="0.01"

```

```

                placeholder="0"
                class="border rounded px-2 py-1 w-32" />
            </td>
        </tr>
    {% endfor %}
</tbody>
</table>

<div class="p-4 flex justify-end gap-2 bg-gray-50">
    <a href="{% url 'dashboard' %}"
        class="px-4 py-2 rounded border">Cancel</a>
    <button type="submit"
        class="bg-green-600 hover:bg-green-700 text-white px-6 py-2 rounded">
        Submit Requisition
    </button>
</div>
</form>
{% endblock %}

```

12.5 form_edit.html

```

{% extends "requisitions/base.html" %}
{% block title %}Edit Requisition{% endblock %}
{% block content %}
<h1 class="text-2xl font-bold mb-4">Edit Requisition – {{ submission.submission_date }}</h1>

<form method="post" class="bg-white shadow rounded-lg overflow-hidden">
    {% csrf_token %}
    <table class="min-w-full text-sm">
        <thead class="bg-yellow-600 text-white">
            <tr>
                <th class="text-left px-4 py-2 w-1/2">Item</th>
                <th class="text-left px-4 py-2">Unit</th>
                <th class="text-left px-4 py-2">Quantity</th>
            </tr>
        </thead>
        <tbody>
            {% for item in items %}
            {% with line=existing|dict_get:item.id %}
                <tr class="border-t">
                    <td class="px-4 py-2 font-medium">{{ item.item_name }}</td>
                    <td class="px-4 py-2">
                        <select name="unit_{{ item.id }}" class="border rounded px-2 py-1 w-full">
                            {% for value,label in units %}
                                <option value="{{ value }}"
                                    {% if line and line.unit == value %}selected{% endif %}>{{ label
                                }}</option>
                            {% endfor %}
                        </select>
                    </td>
                    <td class="px-4 py-2">
                        <input type="number" name="quantity_{{ item.id }}" min="0" step="0.01"

```



```

        value="{% if line %}{{ line.quantity }}{% endif %}"
        class="border rounded px-2 py-1 w-32" />
    </td>
</tr>
{% endwith %}
{% endfor %}
</tbody>
</table>
<div class="p-4 flex justify-end gap-2 bg-gray-50">
    <a href="{% url 'dashboard' %}" class="px-4 py-2 rounded border">Cancel</a>
    <button type="submit"
        class="bg-yellow-600 hover:bg-yellow-700 text-white px-6 py-2 rounded">
        Update Requisition
    </button>
</div>
</form>
{% endblock %}

{# Custom template filter 'dict_get' is required.
Create requisitions/templatetags/dict_extras.py with:
    from django import template
    register = template.Library()
    @register.filter
    def dict_get(d, key):
        return d.get(key)
Load with {% load dict_extras %} at the top of the template. #}

```

12.6 success.html

```

{% extends "requisitions/base.html" %}
{% block title %}Success{% endblock %}
{% block content %}
<div class="bg-white shadow rounded p-8 max-w-2xl mx-auto text-center mt-8">
    <div class="text-5xl text-green-500 mb-4">✓</div>
    <h1 class="text-2xl font-bold mb-2">Requisition Recorded</h1>
    <p class="text-gray-600 mb-6">Submitted on {{ submission.submission_date }} at {{
submission.updated_at|time:"H:i" }}.</p>

    <table class="min-w-full text-sm border mb-6">
        <thead class="bg-gray-100">
            <tr><th class="text-left px-3 py-2">Item</th><th class="text-left px-3
py-2">Qty</th><th class="text-left px-3 py-2">Unit</th></tr>
        </thead>
        <tbody>
            {% for li in submission.line_items.all %}
                <tr class="border-t">
                    <td class="px-3 py-2">{{ li.item.item_name }}</td>
                    <td class="px-3 py-2">{{ li.quantity }}</td>
                    <td class="px-3 py-2">{{ li.unit }}</td>
                </tr>
            {% endfor %}
        </tbody>
    </table>

```

```
<div class="flex justify-center gap-3">
  <a href="{% url 'dashboard' %}" class="bg-blue-600 hover:bg-blue-700 text-white px-4
py-2 rounded">Back to Dashboard</a>
  <a href="{% url 'edit_form' submission.id %}" class="bg-yellow-500 hover:bg-yellow-600
text-white px-4 py-2 rounded">Edit</a>
</div>
</div>
{% endblock %}
```

13. Django Admin Setup

```
# requisitions/admin.py
from django.contrib import admin
from .models import Department, Item, FormSubmission, FormSubmissionItem

@admin.register(Department)
class DepartmentAdmin(admin.ModelAdmin):
    list_display = ("department_name", "user", "created_at")
    search_fields = ("department_name", "user__username")

@admin.register(Item)
class ItemAdmin(admin.ModelAdmin):
    list_display = ("item_name", "is_active", "created_at")
    list_filter = ("is_active",)
    search_fields = ("item_name",)

class LineItemInline(admin.TabularInline):
    model = FormSubmissionItem
    extra = 0

@admin.register(FormSubmission)
class FormSubmissionAdmin(admin.ModelAdmin):
    list_display = ("user", "submission_date", "submitted_at", "updated_at")
    list_filter = ("submission_date",)
    inlines = [LineItemInline]

admin.site.site_header = "Requisition System Admin"
```

Create an admin superuser to access the panel:

```
python manage.py createsuperuser
```

14. Authentication Flow

Django's session middleware and the `@login_required` decorator handle authentication. The flow:

7. User submits credentials on `/login/`.
8. `authenticate()` verifies the hashed password (PBKDF2 by default).
9. `login()` creates a session row and sets the sessionid cookie.
10. Every subsequent request carries the cookie, `AuthenticationMiddleware` attaches `request.user`.
11. Views decorated with `@login_required` automatically redirect anonymous users to `LOGIN_URL`.
12. `logout()` clears the session row and cookie.

14.1 Custom Decorator (optional)

If you want to require a Department record on the user, add this decorator:

```
# requisitions/decorators.py
from functools import wraps
from django.shortcuts import redirect
from django.contrib import messages

def department_required(view_func):
    @wraps(view_func)
    def _wrapped(request, *args, **kwargs):
        if not hasattr(request.user, "department"):
            messages.error(request, "Your account has no department assigned.")
            return redirect("login")
        return view_func(request, *args, **kwargs)
    return _wrapped
```

15. Step-by-Step Setup Instructions

13. Install Python 3.11+ and Node.js 18+.
14. Create project folder and virtual environment, then activate it.
15. Install dependencies: `pip install -r requirements.txt`
16. Run `django-admin startproject requisition_project .` and `python manage.py startapp requisitions`.
17. Add `INSTALLED_APPS` entries (tailwind, theme, requisitions, etc.) in `settings.py`.
18. Run `python manage.py tailwind init theme` then `python manage.py tailwind install`.
19. Paste the `models.py`, `views.py`, `urls.py`, `forms.py`, and templates from sections 6–12.
20. Create migrations: `python manage.py makemigrations requisitions`
21. Apply migrations: `python manage.py migrate`
22. Seed items: `python manage.py seed_items`
23. Create test department: `python manage.py create_test_user`
24. Create admin superuser: `python manage.py createsuperuser`
25. Start Tailwind watcher in one terminal: `python manage.py tailwind start`
26. Start Django dev server in another terminal: `python manage.py runserver`
27. Open `http://127.0.0.1:8000/` and log in.

16. Running the Project

16.1 Development

```
# Terminal 1 – Tailwind compiler (watch mode)
python manage.py tailwind start

# Terminal 2 – Django development server
python manage.py runserver

# App will be available at:
# http://127.0.0.1:8000/      – Login page
# http://127.0.0.1:8000/admin/ – Admin panel
```

16.2 Initialize Database (from scratch)

```
python manage.py makemigrations
python manage.py migrate
python manage.py seed_items
python manage.py create_test_user
python manage.py createsuperuser # creates admin user interactively
```

16.3 Production Build (Tailwind)

```
python manage.py tailwind build
python manage.py collectstatic --noinput
```

16.4 Test Login Credentials

Field	Value
URL	http://127.0.0.1:8000/login/
Username	admin_dept
Password	admin123
Department	Administration

These credentials are created by the `create_test_user` management command. Change them in production.

17. UI Form Layout Reference

The requisition form is rendered as a responsive table with three columns:

Item Name	Unit Dropdown	Quantity
A4 Paper	pcs ▼	[____]
Cello Tape	rolls ▼	[____]
Rice	kg ▼	[____]
Cooking Oil	litres ▼	[____]
Printer Ink (Black)	boxes ▼	[____]

On mobile, Tailwind's responsive utilities allow the table to scroll horizontally inside a wrapper element.

17.1 Page Map

Route	Page	Access
/login/	Login page	Public
/dashboard/	Dashboard with history & action buttons	Authenticated
/form/new/	Create requisition form	Authenticated
/form/<id>/edit/	Edit existing requisition	Authenticated, owner only
/form/<id>/success/	Success confirmation page	Authenticated, owner only
/admin/	Django admin (items, users, submissions)	Staff users only
/logout/	Logout endpoint	Authenticated

18. Validation & Security Summary

- All POST endpoints are protected by Django's CSRF middleware.
- Quantity inputs use type="number" with min="0" and step="0.01" on the frontend, and are re-validated server-side.
- Empty quantity cells are simply skipped — only items with a positive quantity are saved.
- UniqueConstraint at the DB level guarantees a single submission per user per day.
- Items are protected (on_delete=PROTECT) so they cannot be deleted while referenced by historical submissions.
- Passwords are hashed using PBKDF2 (configurable to BCryptSHA256 as shown in settings.py).
- Sessions expire after 8 hours by default; configurable via SESSION_COOKIE_AGE.
- Admin panel restricted to is_staff users.
- DEBUG and SECRET_KEY are read from a .env file — never commit .env to version control.

19. Troubleshooting

Problem	Likely Cause / Fix
Tailwind styles not appearing	Run <code>python manage.py tailwind start</code> ; verify <code>{% tailwind_css %}</code> is in <code>base.html</code> .
TemplateDoesNotExist	Check templates are under <code>requisitions/templates/requisitions/</code> and the app is in <code>INSTALLED_APPS</code> .
UNIQUE constraint failed: ...submission_date	User already has a submission today — redirect to edit instead.
NPM not found (tailwind)	Set <code>NPM_BIN_PATH</code> to the absolute npm path in <code>settings.py</code> .
CSRF verification failed	Ensure <code>{% csrf_token %}</code> is inside every <code><form></code> tag.
No such table	Run <code>python manage.py migrate</code> .
Login redirects in a loop	Confirm <code>LOGIN_REDIRECT_URL</code> points to a view that does not itself require login on failure.

20. Department Master (Admin CRUD)

The Department Master page provides a dedicated admin interface for managing departments and their assigned user accounts, separate from the built-in Django admin panel. It is restricted to users with the `is_staff` flag.

20.1 Why a Custom Department Master?

- Branded UI consistent with the rest of the application (Tailwind styling).
- Single form to create both a User and a linked Department in one transaction.
- Status badges, search, and stat cards optimized for daily use.
- Soft-delete logic — departments with historical submissions are deactivated rather than deleted, preserving the audit trail.

20.2 Add to `forms.py`

```
from django import forms
from django.contrib.auth.models import User
from .models import Department

class DepartmentMasterForm(forms.Form):
    department_name = forms.CharField(
        max_length=120,
        widget=forms.TextInput(attrs={
            "class": "w-full border rounded-lg px-3 py-2",
            "placeholder": "e.g. Finance",
        }),
    )
    username = forms.CharField(
        max_length=150,
        widget=forms.TextInput(attrs={
            "class": "w-full border rounded-lg px-3 py-2",
            "placeholder": "Login username",
        }),
    )
    password = forms.CharField(
        required=False,
        widget=forms.PasswordInput(attrs={
            "class": "w-full border rounded-lg px-3 py-2",
            "placeholder": "Leave blank to keep existing",
        }),
    )
    is_active = forms.BooleanField(required=False, initial=True)

    def __init__(self, *args, instance=None, **kwargs):
        self.instance = instance
        super().__init__(*args, **kwargs)
        if instance:
            self.fields["department_name"].initial = instance.department_name
            self.fields["username"].initial = instance.user.username
```

```

        self.fields["is_active"].initial = instance.user.is_active

def clean_department_name(self):
    name = self.cleaned_data["department_name"].strip()
    qs = Department.objects.filter(department_name__iexact=name)
    if self.instance:
        qs = qs.exclude(pk=self.instance.pk)
    if qs.exists():
        raise forms.ValidationError("A department with this name already exists.")
    return name

def clean_username(self):
    username = self.cleaned_data["username"].strip()
    qs = User.objects.filter(username__iexact=username)
    if self.instance:
        qs = qs.exclude(pk=self.instance.user.pk)
    if qs.exists():
        raise forms.ValidationError("This username is already taken.")
    return username

def clean(self):
    cleaned = super().clean()
    if not self.instance and not cleaned.get("password"):
        self.add_error("password", "Password is required for new departments.")
    return cleaned

```

20.3 Add to views.py

```

from django.contrib.auth.decorators import login_required, user_passes_test
from django.contrib.auth.models import User
from django.db.models import Q
from .forms import DepartmentMasterForm

def is_staff(user):
    return user.is_authenticated and user.is_staff

@login_required
@user_passes_test(is_staff)
def department_list(request):
    q = request.GET.get("q", "").strip()
    status = request.GET.get("status", "all")

    qs = Department.objects.select_related("user").order_by("department_name")
    if q:
        qs = qs.filter(
            Q(department_name__icontains=q) | Q(user__username__icontains=q)
        )
    if status == "active":
        qs = qs.filter(user__is_active=True)
    elif status == "inactive":
        qs = qs.filter(user__is_active=False)

```

```

paginator = Paginator(qs, 15)
page = paginator.get_page(request.GET.get("page"))

stats = {
    "total": Department.objects.count(),
    "active": Department.objects.filter(user__is_active=True).count(),
    "inactive": Department.objects.filter(user__is_active=False).count(),
}

return render(request, "requisitions/department_list.html", {
    "page_obj": page, "q": q, "status": status, "stats": stats,
})

@login_required
@user_passes_test(is_staff)
def department_create(request):
    if request.method == "POST":
        form = DepartmentMasterForm(request.POST)
        if form.is_valid():
            with transaction.atomic():
                user = User.objects.create_user(
                    username=form.cleaned_data["username"],
                    password=form.cleaned_data["password"],
                )
                user.is_active = form.cleaned_data.get("is_active", True)
                user.save()
                Department.objects.create(
                    user=user,
                    department_name=form.cleaned_data["department_name"],
                )
            messages.success(request, "Department created.")
            return redirect("department_list")
    else:
        form = DepartmentMasterForm()
    return render(request, "requisitions/department_form.html", {
        "form": form, "title": "Add department",
    })

@login_required
@user_passes_test(is_staff)
def department_edit(request, dept_id):
    department = get_object_or_404(Department, id=dept_id)
    if request.method == "POST":
        form = DepartmentMasterForm(request.POST, instance=department)
        if form.is_valid():
            with transaction.atomic():
                department.department_name = form.cleaned_data["department_name"]
                department.save()
                user = department.user
                user.username = form.cleaned_data["username"]
                user.is_active = form.cleaned_data.get("is_active", True)
                if form.cleaned_data.get("password"):

```

```

        user.set_password(form.cleaned_data["password"])
        user.save()
        messages.success(request, "Department updated.")
        return redirect("department_list")
    else:
        form = DepartmentMasterForm(instance=department)
    return render(request, "requisitions/department_form.html", {
        "form": form, "title": f"Edit {department.department_name}",
        "department": department,
    })

@login_required
@user_passes_test(is_staff)
def department_delete(request, dept_id):
    department = get_object_or_404(Department, id=dept_id)
    if request.method == "POST":
        if department.user.submissions.exists():
            department.user.is_active = False
            department.user.save()
            messages.info(request, "Department deactivated (has historical submissions).")
        else:
            user = department.user
            department.delete()
            user.delete()
            messages.success(request, "Department deleted.")
        return redirect("department_list")
    return render(request, "requisitions/department_confirm_delete.html", {
        "department": department,
    })

```

20.4 Add to urls.py

```

path("departments/",
     views.department_list, name="department_list"),
path("departments/new/",
     views.department_create, name="department_create"),
path("departments/<int:dept_id>/edit/",
     views.department_edit, name="department_edit"),
path("departments/<int:dept_id>/delete/",
     views.department_delete, name="department_delete"),

```

20.5 Template — department_list.html

```

{% extends "requisitions/base.html" %}
{% block title %}Department Master{% endblock %}
{% block content %}
<div class="flex justify-between items-center mb-4">
  <div>
    <h1 class="text-2xl font-bold">Department Master</h1>
    <p class="text-sm text-gray-500">Manage all departments and their assigned users</p>
  </div>

```

```

<a href="{% url 'department_create' %}"
  class="bg-green-700 hover:bg-green-800 text-white px-4 py-2 rounded">+ Add
Department</a>
</div>

<div class="grid grid-cols-3 gap-3 mb-4">
  <div class="bg-white border rounded p-3">
    <div class="text-xs text-gray-500">Total</div>
    <div class="text-xl font-semibold">{{ stats.total }}</div>
  </div>
  <div class="bg-white border rounded p-3">
    <div class="text-xs text-gray-500">Active</div>
    <div class="text-xl font-semibold text-green-700">{{ stats.active }}</div>
  </div>
  <div class="bg-white border rounded p-3">
    <div class="text-xs text-gray-500">Inactive</div>
    <div class="text-xl font-semibold text-gray-500">{{ stats.inactive }}</div>
  </div>
</div>

<form method="get" class="bg-white border rounded p-3 mb-3 flex gap-2">
  <input type="text" name="q" value="{{ q }}"
    placeholder="Search department or username..."
    class="flex-1 border rounded px-3 py-1.5 text-sm" />
  <select name="status" class="border rounded px-3 py-1.5 text-sm">
    <option value="all"      {% if status == "all" %}>selected{% endif %}>All
status</option>
    <option value="active"   {% if status == "active" %}>selected{% endif %}>Active</option>
    <option value="inactive" {% if status == "inactive" %}>selected{% endif
%}>Inactive</option>
  </select>
  <button class="bg-blue-700 text-white px-4 rounded text-sm">Filter</button>
</form>

<div class="bg-white border rounded overflow-hidden">
  <table class="min-w-full text-sm">
    <thead class="bg-gray-100 text-left">
      <tr>
        <th class="px-4 py-2">ID</th>
        <th class="px-4 py-2">Department</th>
        <th class="px-4 py-2">Username</th>
        <th class="px-4 py-2">Created</th>
        <th class="px-4 py-2">Status</th>
        <th class="px-4 py-2 text-right">Actions</th>
      </tr>
    </thead>
    <tbody>
      {% for d in page_obj %}
      <tr class="border-t">
        <td class="px-4 py-2 font-mono text-gray-500">{{ d.id|stringformat:"02d" }}</td>
        <td class="px-4 py-2 font-medium">{{ d.department_name }}</td>
        <td class="px-4 py-2 font-mono text-xs">{{ d.user.username }}</td>
        <td class="px-4 py-2 text-gray-500">{{ d.created_at|date:"d M Y" }}</td>
        <td class="px-4 py-2">
          {% if d.user.is_active %}

```

```

        <span class="bg-green-100 text-green-800 px-2 py-0.5 rounded-full
text-xs">active</span>
        {% else %}
        <span class="bg-gray-100 text-gray-600 px-2 py-0.5 rounded-full
text-xs">inactive</span>
        {% endif %}
    </td>
    <td class="px-4 py-2 text-right">
        <a href="{% url 'department_edit' d.id %}"
        class="text-blue-700 hover:underline mr-3">Edit</a>
        <a href="{% url 'department_delete' d.id %}"
        class="text-red-700 hover:underline">Delete</a>
    </td>
</tr>
{% empty %}
<tr><td colspan="6" class="text-center text-gray-500 py-6">No departments
found.</td></tr>
{% endfor %}
</tbody>
</table>
</div>
{% endblock %}

```

20.6 Template — department_form.html

```

{% extends "requisitions/base.html" %}
{% block content %}
<div class="max-w-xl mx-auto bg-white border rounded-lg p-6">
  <h1 class="text-xl font-bold mb-4">{{ title }}</h1>
  <form method="post" class="space-y-4">
    {% csrf_token %}
    <div>
      <label class="block text-sm font-medium mb-1">Department name</label>
      {{ form.department_name }}
      {% for e in form.department_name.errors %}
        <p class="text-red-600 text-xs mt-1">{{ e }}</p>
      {% endfor %}
    </div>
    <div>
      <label class="block text-sm font-medium mb-1">Username</label>
      {{ form.username }}
      {% for e in form.username.errors %}
        <p class="text-red-600 text-xs mt-1">{{ e }}</p>
      {% endfor %}
    </div>
    <div>
      <label class="block text-sm font-medium mb-1">Password</label>
      {{ form.password }}
      {% for e in form.password.errors %}
        <p class="text-red-600 text-xs mt-1">{{ e }}</p>
      {% endfor %}
    </div>
    <label class="flex items-center gap-2">

```

```

    {{ form.is_active }}
    <span class="text-sm">Active</span>
</label>
<div class="flex justify-end gap-2">
  <a href="{% url 'department_list' %}" class="px-4 py-2 border rounded">Cancel</a>
  <button class="bg-green-700 hover:bg-green-800 text-white px-4 py-2
rounded">Save</button>
</div>
</form>
</div>
{% endblock %}

```

20.7 Design Notes

- Both User and Department are created or updated inside `transaction.atomic()` so a failure on either rolls back the whole operation.
- Soft-delete: if the department has historical submissions, only the user's `is_active` flag is set to `False` — the records remain intact.
- Hard-delete is used only when there are no submissions, in which case both Department and User rows are removed.
- Access is gated by `@user_passes_test(is_staff)`; non-staff authenticated users get redirected to `/login/`.

21. Item Master (Admin CRUD)

The Item Master page lets administrators manage the fixed list of items that appear on every requisition form. It introduces two enhancements to the original Item model: a category field for grouping, and a default_unit field that pre-selects the unit dropdown on the requisition form.

21.1 Extend the Item Model

Replace the Item model definition in models.py with the version below, then run makemigrations and migrate.

```
class Item(models.Model):
    CATEGORY_CHOICES = [
        ("stationery", "Stationery"),
        ("it_supplies", "IT Supplies"),
        ("hygiene", "Hygiene"),
        ("pantry", "Pantry"),
        ("misc", "Miscellaneous"),
    ]

    UNIT_CHOICES = [
        ("pcs", "pcs"), ("rolls", "rolls"), ("kg", "kg"), ("g", "g"),
        ("litres", "litres"), ("ml", "ml"), ("boxes", "boxes"),
        ("packets", "packets"), ("dozen", "dozen"), ("metres", "metres"),
    ]

    item_name = models.CharField(max_length=150, unique=True)
    category = models.CharField(max_length=20, choices=CATEGORY_CHOICES,
default="misc")
    default_unit = models.CharField(max_length=20, choices=UNIT_CHOICES, default="pcs")
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ["category", "item_name"]

    def __str__(self):
        return self.item_name
```

```
python manage.py makemigrations requisitions
python manage.py migrate
```

21.2 Add to forms.py

```
from .models import Item

class ItemMasterForm(forms.ModelForm):
    class Meta:
        model = Item
        fields = ["item_name", "category", "default_unit", "is_active"]
```

```

widgets = {
    "item_name": forms.TextInput(attrs={
        "class": "w-full border rounded-lg px-3 py-2",
        "placeholder": "e.g. A4 Paper",
    }),
    "category": forms.Select(attrs={"class": "w-full border rounded-lg px-3
py-2"}),
    "default_unit": forms.Select(attrs={"class": "w-full border rounded-lg px-3
py-2"}),
    "is_active": forms.CheckboxInput(attrs={"class": "h-4 w-4"}),
}

def clean_item_name(self):
    name = self.cleaned_data["item_name"].strip()
    qs = Item.objects.filter(item_name__iexact=name)
    if self.instance.pk:
        qs = qs.exclude(pk=self.instance.pk)
    if qs.exists():
        raise forms.ValidationError("An item with this name already exists.")
    return name

```

21.3 Add to views.py

```

from .forms import ItemMasterForm

@login_required
@user_passes_test(is_staff)
def item_list(request):
    q = request.GET.get("q", "").strip()
    category = request.GET.get("category", "all")
    status = request.GET.get("status", "all")

    qs = Item.objects.all()
    if q:
        qs = qs.filter(item_name__icontains=q)
    if category != "all":
        qs = qs.filter(category=category)
    if status == "active":
        qs = qs.filter(is_active=True)
    elif status == "retired":
        qs = qs.filter(is_active=False)

    paginator = Paginator(qs, 20)
    page = paginator.get_page(request.GET.get("page"))

    stats = {
        "total": Item.objects.count(),
        "active": Item.objects.filter(is_active=True).count(),
        "retired": Item.objects.filter(is_active=False).count(),
        "categories": Item.objects.values("category").distinct().count(),
    }

```

```

        return render(request, "requisitions/item_list.html", {
            "page_obj": page, "q": q, "category": category, "status": status,
            "stats": stats, "category_choices": Item.CATEGORY_CHOICES,
        })

@login_required
@user_passes_test(is_staff)
def item_create(request):
    if request.method == "POST":
        form = ItemMasterForm(request.POST)
        if form.is_valid():
            form.save()
            messages.success(request, "Item added.")
            return redirect("item_list")
    else:
        form = ItemMasterForm()
    return render(request, "requisitions/item_form.html", {
        "form": form, "title": "Add item",
    })

@login_required
@user_passes_test(is_staff)
def item_edit(request, item_id):
    item = get_object_or_404(Item, id=item_id)
    if request.method == "POST":
        form = ItemMasterForm(request.POST, instance=item)
        if form.is_valid():
            form.save()
            messages.success(request, "Item updated.")
            return redirect("item_list")
    else:
        form = ItemMasterForm(instance=item)
    return render(request, "requisitions/item_form.html", {
        "form": form, "title": f"Edit {item.item_name}", "item": item,
    })

@login_required
@user_passes_test(is_staff)
def item_delete(request, item_id):
    item = get_object_or_404(Item, id=item_id)
    if request.method == "POST":
        if FormSubmissionItem.objects.filter(item=item).exists():
            item.is_active = False
            item.save()
            messages.info(request, "Item retired (has historical usage; cannot hard-delete).")
        else:
            item.delete()
            messages.success(request, "Item deleted.")
            return redirect("item_list")
    return render(request, "requisitions/item_confirm_delete.html", {"item": item})

```

```

@login_required
@user_passes_test(is_staff)
def item_bulk_import(request):
    if request.method == "POST":
        csv_file = request.FILES.get("csv_file")
        if not csv_file:
            messages.error(request, "No file uploaded.")
            return redirect("item_bulk_import")

        import csv, io
        decoded = csv_file.read().decode("utf-8")
        reader = csv.DictReader(io.StringIO(decoded))
        created, skipped = 0, 0
        for row in reader:
            name = (row.get("item_name") or "").strip()
            if not name:
                continue
            obj, was_created = Item.objects.get_or_create(
                item_name=name,
                defaults={
                    "category": (row.get("category") or "misc").strip(),
                    "default_unit": (row.get("default_unit") or "pcs").strip(),
                    "is_active": True,
                },
            )
            if was_created: created += 1
            else: skipped += 1
        messages.success(request,
            f"Imported {created} new items, {skipped} skipped (duplicates).")
        return redirect("item_list")
    return render(request, "requisitions/item_bulk_import.html")

```

21.4 Add to urls.py

```

path("items/",
     views.item_list,      name="item_list"),
path("items/new/",
     views.item_create,    name="item_create"),
path("items/<int:item_id>/edit/",
     views.item_edit,      name="item_edit"),
path("items/<int:item_id>/delete/",
     views.item_delete,    name="item_delete"),
path("items/import/",
     views.item_bulk_import, name="item_bulk_import"),

```

21.5 Template — item_list.html

```

{% extends "requisitions/base.html" %}
{% block title %}Item Master{% endblock %}
{% block content %}
<div class="flex justify-between items-center mb-4">

```

```

<div>
  <h1 class="text-2xl font-bold">Item Master</h1>
  <p class="text-sm text-gray-500">Fixed list of items shown on every requisition
form</p>
</div>
<div class="flex gap-2">
  <a href="{% url 'item_bulk_import' %}"
    class="border bg-white px-4 py-2 rounded">Bulk Import</a>
  <a href="{% url 'item_create' %}"
    class="bg-green-700 hover:bg-green-800 text-white px-4 py-2 rounded">+ Add Item</a>
</div>
</div>

<div class="grid grid-cols-4 gap-3 mb-4">
  <div class="bg-white border rounded p-3">
    <div class="text-xs text-gray-500">Total</div>
    <div class="text-xl font-semibold">{{ stats.total }}</div>
  </div>
  <div class="bg-white border rounded p-3">
    <div class="text-xs text-gray-500">Active</div>
    <div class="text-xl font-semibold text-green-700">{{ stats.active }}</div>
  </div>
  <div class="bg-white border rounded p-3">
    <div class="text-xs text-gray-500">Retired</div>
    <div class="text-xl font-semibold text-gray-500">{{ stats.retired }}</div>
  </div>
  <div class="bg-white border rounded p-3">
    <div class="text-xs text-gray-500">Categories</div>
    <div class="text-xl font-semibold text-blue-700">{{ stats.categories }}</div>
  </div>
</div>

<form method="get" class="bg-white border rounded p-3 mb-3 flex gap-2">
  <input type="text" name="q" value="{{ q }}" placeholder="Search item name..."
    class="flex-1 border rounded px-3 py-1.5 text-sm" />
  <select name="category" class="border rounded px-3 py-1.5 text-sm">
    <option value="all">All categories</option>
    {% for value, label in category_choices %}
      <option value="{{ value }}" {% if category == value %}selected{% endif %}>{{ label
}}</option>
    {% endfor %}
  </select>
  <select name="status" class="border rounded px-3 py-1.5 text-sm">
    <option value="all" {% if status == "all" %}selected{% endif %}>All status</option>
    <option value="active" {% if status == "active" %}selected{% endif %}>Active</option>
    <option value="retired" {% if status == "retired" %}selected{% endif
%}>Retired</option>
  </select>
  <button class="bg-blue-700 text-white px-4 rounded text-sm">Filter</button>
</form>

<div class="bg-white border rounded overflow-hidden">
  <table class="min-w-full text-sm">
    <thead class="bg-gray-100 text-left">
      <tr>

```

```

        <th class="px-4 py-2">ID</th>
        <th class="px-4 py-2">Item</th>
        <th class="px-4 py-2">Category</th>
        <th class="px-4 py-2">Default Unit</th>
        <th class="px-4 py-2">Status</th>
        <th class="px-4 py-2 text-right">Actions</th>
    </tr>
</thead>
<tbody>
    {% for item in page_obj %}
    <tr class="border-t">
        <td class="px-4 py-2 font-mono text-gray-500">{{ item.id|stringformat:"03d" }}</td>
        <td class="px-4 py-2 font-medium">{{ item.item_name }}</td>
        <td class="px-4 py-2">
            <span class="bg-blue-100 text-blue-800 px-2 py-0.5 rounded-full text-xs">
                {{ item.get_category_display }}
            </span>
        </td>
        <td class="px-4 py-2 font-mono text-xs">{{ item.default_unit }}</td>
        <td class="px-4 py-2">
            {% if item.is_active %}
            <span class="bg-green-100 text-green-800 px-2 py-0.5 rounded-full
text-xs">active</span>
            {% else %}
            <span class="bg-gray-100 text-gray-600 px-2 py-0.5 rounded-full
text-xs">retired</span>
            {% endif %}
        </td>
        <td class="px-4 py-2 text-right">
            <a href="{% url 'item_edit' item.id %}"
                class="text-blue-700 hover:underline mr-3">Edit</a>
            <a href="{% url 'item_delete' item.id %}"
                class="text-red-700 hover:underline">Delete</a>
        </td>
    </tr>
    {% empty %}
    <tr><td colspan="6" class="text-center text-gray-500 py-6">No items found.</td></tr>
    {% endfor %}
</tbody>
</table>
</div>
{% endblock %}

```

21.6 Template — item_form.html

```

{% extends "requisitions/base.html" %}
{% block content %}
<div class="max-w-xl mx-auto bg-white border rounded-lg p-6">
    <h1 class="text-xl font-bold mb-4">{{ title }}</h1>
    <form method="post" class="space-y-4">
        {% csrf_token %}
        <div>
            <label class="block text-sm font-medium mb-1">Item name</label>

```

```

    {{ form.item_name }}
    {% for e in form.item_name.errors %}
      <p class="text-red-600 text-xs mt-1">{{ e }}</p>
    {% endfor %}
  </div>
  <div>
    <label class="block text-sm font-medium mb-1">Category</label>
    {{ form.category }}
  </div>
  <div>
    <label class="block text-sm font-medium mb-1">Default unit</label>
    {{ form.default_unit }}
    <p class="text-xs text-gray-500 mt-1">
      Pre-selected on the requisition form, but users can change it.
    </p>
  </div>
  <label class="flex items-center gap-2">
    {{ form.is_active }}
    <span class="text-sm">Active (visible on requisition form)</span>
  </label>
  <div class="flex justify-end gap-2 pt-2">
    <a href="{% url 'item_list' %}" class="px-4 py-2 border rounded">Cancel</a>
    <button class="bg-green-700 hover:bg-green-800 text-white px-4 py-2
rounded">Save</button>
  </div>
</form>
</div>
{% endblock %}

```

21.7 Template — item_bulk_import.html

```

{% extends "requisitions/base.html" %}
{% block content %}
  <div class="max-w-xl mx-auto bg-white border rounded-lg p-6">
    <h1 class="text-xl font-bold mb-2">Bulk Import Items</h1>
    <p class="text-sm text-gray-600 mb-4">
      Upload a CSV with columns: item_name, category, default_unit.
      Duplicates (by name) are skipped.
    </p>
    <form method="post" enctype="multipart/form-data" class="space-y-4">
      {% csrf_token %}
      <input type="file" name="csv_file" accept=".csv" required
        class="block w-full text-sm" />
      <div class="flex justify-end gap-2">
        <a href="{% url 'item_list' %}" class="px-4 py-2 border rounded">Cancel</a>
        <button class="bg-blue-700 hover:bg-blue-800 text-white px-4 py-2
rounded">Import</button>
      </div>
    </form>

    <div class="mt-6">
      <p class="text-xs text-gray-500 mb-1">Example CSV:</p>

```

```

    <pre class="bg-gray-50 border rounded p-3 text-xs
font-mono">item_name,category,default_unit
A4 Paper,stationery,pcs
Cello Tape,stationery,rolls
Rice,pantry,kg
Hand Sanitizer,hygiene,litres</pre>
  </div>
</div>
{% endblock %}

```

21.8 Update the Requisition Form

In `form_create.html`, update the unit dropdown so each item pre-selects its default unit:

```

<select name="unit_{{ item.id }}" class="border rounded px-2 py-1 w-full">
  {% for value,label in units %}
    <option value="{{ value }}"
      {% if value == item.default_unit %}selected{% endif %}>{{ label }}</option>
  {% endfor %}
</select>

```

21.9 Design Notes

- Soft-delete: items referenced by `FormSubmissionItem` cannot be hard-deleted (`on_delete=PROTECT`). The delete view retires them instead, hiding them from the form while preserving history.
- Bulk import uses `get_or_create` so re-running the same CSV is idempotent — duplicates are skipped, not overwritten.
- The category field groups related items in the form and enables filtered views in the master.
- `default_unit` removes a click for the common case: A4 Paper defaults to pcs, Rice to kg, Hand Sanitizer to litres.

22. Updated Page Map

Complete list of routes after adding the Department Master and Item Master sections.

Route	Page	Access
/login/	Login page	Public
/dashboard/	Dashboard with history	Authenticated
/form/new/	Create requisition form	Authenticated
/form/<id>/edit/	Edit existing requisition	Authenticated, owner only
/form/<id>/success/	Success confirmation	Authenticated, owner only
/departments/	Department master list	Staff only
/departments/new/	Add department	Staff only
/departments/<id>/edit/	Edit department	Staff only
/departments/<id>/delete/	Delete or deactivate department	Staff only
/items/	Item master list	Staff only
/items/new/	Add item	Staff only
/items/<id>/edit/	Edit item	Staff only
/items/<id>/delete/	Delete or retire item	Staff only
/items/import/	Bulk CSV import	Staff only
/admin/	Django admin (full DB access)	Superuser only
/logout/	Logout endpoint	Authenticated

23. Appendix — Node.js vs Django Mapping

For readers familiar with the original Node.js specification, this table maps each Node concept to its Django equivalent.

Node.js / Express	Django Equivalent
express-generator scaffold	django-admin startproject + startapp
app.js / server.js	manage.py + wsgi.py
routes/ directory	urls.py (project + app level)
controllers/ directory	views.py (function or class based)
models/ with raw SQL	models.py with Django ORM
EJS templates (.ejs)	Django templates (.html) with DTL
express-session	django.contrib.sessions
bcrypt	PBKDF2 (default) or BCryptSHA256 hasher
connect-flash	django.contrib.messages
body-parser / multer	request.POST / request.FILES (built-in)
sqlite3 npm package	django.db.backends.sqlite3
Manual middleware	Django MIDDLEWARE setting
No built-in admin	django.contrib.admin (free)

End of document.